

ORACLE 1Z0-1157-26 FOUNDATIONS ASSOCIATE

Module 4 Complete Exam Study Notes: OpenAI Responses API & Agents SDK

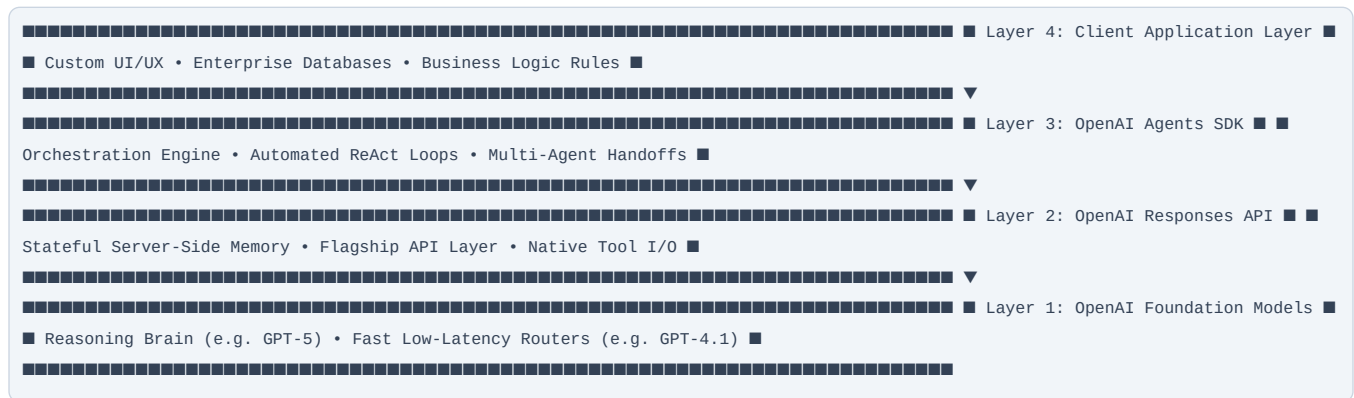
AGENT SDK

RESPONSES API

OPEN SOURCE STUDY NOTE

1. Core Foundations & The 4-Layer Hierarchy

The **OpenAI Agent Stack** structures agentic AI systems into four distinct vertical layers, modularizing how developers connect core model capabilities into client-facing applications [cite: 84]. Understanding the precise responsibilities of each layer and the strict chain of control is critical for the **Oracle 1Z0-1157-26 exam** [cite: 84, 85].

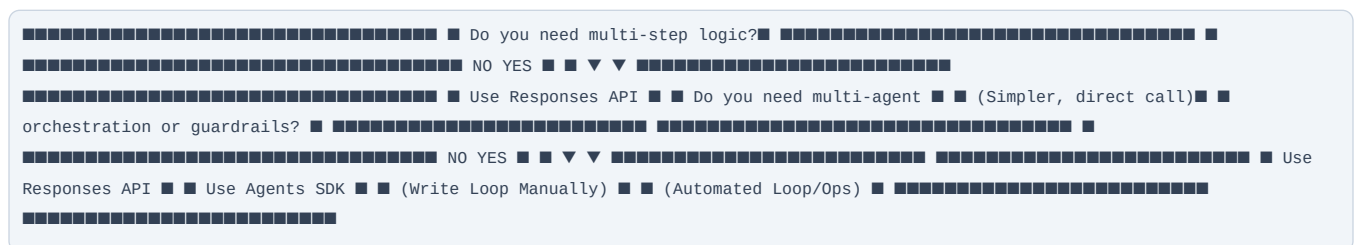


The Four Layers Explained:

- **Layer 1 — Foundation Models**: The raw cognitive engine [cite: 84]. Splits into high-capacity, step-by-step *Reasoning Models* (optimized for deep multi-step logical planning and math) and lightweight *Fast Non-Reasoning Models* (optimized for low-latency triage and perimeter guardrail checks) [cite: 84]. Flagship implementations target models like **gpt-5.5** [cite: 84].
- **Layer 2 — Responses API**: OpenAI's flagship, core foundational interface [cite: 87, 90]. It is natively stateful, supporting conversational chaining without client-side message serialization [cite: 90]. Directly exposes single-line platform tool registrations and function-calling specifications [cite: 90].
- **Layer 3 — Agents SDK**: A high-level Python framework (openai-agents) built directly on top of the Responses API [cite: 84]. It abstracts loop mechanics, managing multi-agent handoffs, guardrails, and Tracing [cite: 84].
- **Layer 4 — Application Layer**: The host application code hosting the customer UI, custom databases, CRM systems, and environment configurations [cite: 84].

Responses API vs. Agents SDK Selection Logic

The primary architectural distinction between the two middle tiers centers on **Loop Management (Nervous System)** [cite: 83, 85]. With the low-level **Responses API**, the developer holds direct control but must manually write and manage the ReAct execution loop, handling retries and state transitions [cite: 82, 85]. With the **Agents SDK**, the framework automates and handles the ReAct loop out of the box using built-in runners [cite: 82, 85].



EXAM ANALOGY — Building vs. Driving: Using the **Responses API** directly is like *building a car engine from scratch*; it provides ultimate control over every component but requires heavy development [cite: 83, 85]. Using the **Agents SDK** is like *driving a car with a pre-built engine, steering, and safety systems*; the manual operational plumbing is entirely handled for you [cite: 83, 85].

2. OpenAI Responses API (Layer 2 Deep Dive)

Released in March 2025, the **Responses API** is OpenAI's flagship interface [cite: 87, 90]. It unifies and replaces two older, deprecated execution paths: the stateless **Chat Completions API** (which required heavy, repetitive payload transmission) and the heavy, rigid **Assistants API** [cite: 90].

Core Strategic Advancements:

- 1. **Simpler Input & Direct Output:** Replaces complex multi-nested structures with a direct string input. Result content is retrieved cleanly via the direct response.output_text property, completely eliminating legacy dictionary traversals like response.choices[0].message.content [cite: 87, 90].
- 2. **Server-Side Conversation Chaining:** Enables state retention natively on OpenAI's servers by passing the previous_response_id pointer of the prior turn [cite: 88, 90]. This creates a back-referencing chain without requiring the developer to re-upload the entire accumulated token history on every turn [cite: 88, 90].
- 3. **1-Line Platform Tools:** Enables native hosted tools (such as live Web Search, File Search, and Code Interpreter) with a single declarative configuration line, running completely inside OpenAI's sandboxed platform without local environment scripting [cite: 88, 91].

Architectural Metric	Legacy Chat Completions API	Modern Responses API
State Location	Client-Side Only (Entire history serialized and sent on every turn)[cite: 88, 90].	Server-Side (Handled via previous_response_id pointers)[cite: 88, 90].
Network Overhead	Linear token expansion O(N); heavy payload inflation[cite: 88, 90].	Constant size; only new prompts and IDs are transmitted[cite: 88, 90].
Output Extraction	Traverses deep arrays: choices[0].message.content[cite: 87, 90].	Direct top-level property: response.output_text[cite: 87, 90].
Built-in Tools	Requires custom schema definitions and manual tool loop handling[cite: 90].	1-line declarative array flags (e.g. {"type": "web_search"})[cite: 88, 91].
Large Tool Data	Large tool observations must bounce across client pipes[cite: 90].	Retained and resolved server-side through pointer links[cite: 90].

Conversation Chaining Code Walkthrough:

```
import os
from openai import OpenAI

client = OpenAI() # Safely reads OPENAI_API_KEY from environment

# Turn 1: Seeding Context Server-Side
response_1 = client.responses.create(
    model="gpt-4.1",
    input="My name is Alice and I love hiking."
)
print(f"Turn 1 Output: {response_1.output_text}")

# Turn 2: Referencing Context via previous_response_id Pointer
response_2 = client.responses.create(
    model="gpt-4.1",
    previous_response_id=response_1.id, # Server walks back chain to resolve state
    input="What is my name and hobby?"
)
print(f"Turn 2 Output: {response_2.output_text}")
# Output: Alice, and your hobby is hiking.
```

EXAM ALERT — Built-in Tool Autonomy: Enabling platform tools like {"type": "web_search"} in the tools array does NOT force their execution [cite: 89, 91]. The foundational model determines autonomy: if a user asks a static query (e.g. 'What is the capital of France?'), the model bypasses the tool [cite: 89, 91]. If the prompt requires fresh details (e.g. 'AI news this week'), the model dynamically dispatches web search [cite: 89, 91].

3. OpenAI Agents SDK (Layer 3 & Tool Architecture)

The **OpenAI Agents SDK** is a lightweight, opinionated Python framework built directly on top of the Responses API [cite: 84]. It consolidates the three core primitives of an agent—the LLM (Brain), Tools (Hands), and the Loop (Nervous System)—into an automated, cohesive system managed by a runtime Runner engine [cite: 83, 85].

The Three SDK Tool Categories:

- 1. **Hosted Tools (Platform Managed):** Pre-built, managed services provided directly by OpenAI [cite: 96, 99]. They require zero local execution plumbing or security boundary setup [cite: 96, 99]. Features include `web_search`, `file_search` (RAG), and `code_interpreter` [cite: 96, 99].
- 2. **Function Tools (Local Custom Logic):** Custom functions executed locally on the host environment [cite: 96, 99]. Decorated with the `@function_tool` decorator, the SDK automatically parses Python metadata to compile OpenAI-compatible JSON schemas [cite: 96, 99].
- 3. **Agents as Tool (Hierarchical Delegation):** Casting an entire agent into a callable tool using `agent.as_tool` [cite: 96, 99]. This allows a high-level manager agent to delegate complex sub-tasks to specialized subordinate worker agents [cite: 96, 99].

Anatomy of `@function_tool` Schema Compilation:

When you decorate a standard Python function, three code elements are extracted by the SDK to generate instructions for the LLM [cite: 96, 99]:

Python Code Element	Compiled JSON Schema Mapping	Instructional Role for the AI Agent
Function Decorator (<code>@function_tool</code>)	Registers function name as a candidate choice.	Signals to the LLM that this deterministic action is available[cite: 96, 99].
Type Hints (e.g. <code>a: float, b: float</code>)	Translated to <code>inputSchema</code> properties: <code>type</code> and <code>required</code> .	Provides strict mathematical data types to validate inputs[cite: 70, 73, 96].
Docstrings (Triple-Quoted String)	Mapped to the <code>'description'</code> field in the schema.	The semantic prompt read by the model to decide WHEN and WHY to call it[cite: 70, 73, 96].
Return Type Hint (<code>-> float</code>)	Exposes output type constraints.	Allows the model to plan how to pass one tool's output to another[cite: 71, 73].

Hands-on Local Tool Definition & Registration:

```
from agents import Agent, Runner, function_tool

# Custom Function Tool definitions (Type Hints & Docstrings are MANDATORY)
@function_tool
def multiply(a: float, b: float) -> float:
    """Multiply two floating-point numbers together. Use for multiplication."""
    return a * b

@function_tool
def divide(a: float, b: float) -> float:
    """Divide number a by b. Use for division operations."""
    if b == 0.0:
        return "error, division by zero"
    return a / b

# Registering tools inside the Agent definition
math_agent = Agent(
    name="Math Specialist",
    instructions="You are a precise math assistant. Always use tools to calculate.",
    tools=[multiply, divide], # Compiles and attaches schemas automatically
    model="gpt-5.5"
)
```

EXAM ALERT — The Security Execution Boundary: A fundamental OCI objective is recognizing that **the LLM never executes tools or runs Python code itself** [cite: 70, 73, 96]. It only generates a structured text string requesting an action (JSON-RPC) [cite: 23, 25]. The local application hosting environment retains the execution boundary: it intercepts the JSON-RPC request, maps the string to its internal registry, safely runs the local code, and passes the output observation back to the stateless LLM [cite: 24, 25, 31, 32].

4. Multi-Agent Systems & Handoff Mechanics

When an operational goal is too complex for a single agent to manage efficiently, the OpenAI Agent Stack implements multi-agent patterns [cite: 101, 102]. Partitioning concerns into small, specialized agents drastically improves modularity, reduces token context clutter, and scales enterprise governance [cite: 101, 102].

Architectural Patterns: Decentralized Handoffs vs. Centralized Manager

The Agents SDK supports two primary multi-agent interaction paradigms [cite: 101, 102]:

Pattern 1: Handoffs Pattern (Decentralized routing) User [] [Triage Agent] [](Handoff control)[] [Specialist Agent] []
Direct Final Answer (Transfers full context) (Takes full control) Pattern 2: Manager Pattern (Centralized coordination) User [] [Manager Agent] [](Calls as tool)[] [Worker Agent 1] [](Returns result)[] [](Calls as tool)[] [Worker Agent 2] [](Returns result)[] ▼ Synthesizes unified reply

Architectural Dimension	Decentralized Handoffs Pattern	Centralized Manager Pattern
Control Model	Decentralized: The handing-off agent completely relinquishes control to the target specialist[cite: 103].	Centralized: The orchestrating manager retains continuous control over execution[cite: 103].
Delegation Mechanic	Conversation control and context are routed via direct handoffs[cite: 103].	Worker agents are loaded and called as standard tools (agent.as_tool)[cite: 96, 99, 103].
Context Flow	Specialist receives the entire conversation history from the session[cite: 103].	Manager selectively passes parameters and aggregates worker replies[cite: 103].
Common Use Case	Domain-routed customer support (Billing, Tech, Shipping triage)[cite: 102, 103].	Complex planning tasks requiring cross-specialist coordination (PM, Writer)[cite: 103].

Code Walkthrough: Customer Handoff System

```
from agents import Agent, Runner

# Specialist Agents with handoff descriptions
math_tutor = Agent(
    name="Math Tutor",
    instructions="Solve math issues cleanly.",
    handoff_description="Handles mathematical queries, arithmetic, and percentages.",
    model="gpt-5.5"
)

history_tutor = Agent(
    name="History Tutor",
    instructions="Provide rich historical context.",
    handoff_description="Handles history queries, civilizations, dates, and historical figures.",
    model="gpt-5.5"
)

# Frontline Triage Agent linking specialists
triage_agent = Agent(
    name="Triage Specialist",
    instructions="Examine input and route to the correct Specialist tutor.",
    handoffs=[math_tutor, history_tutor], # Compiles handoffs into model-callable tools
    model="gpt-5.5"
)

result = Runner.run_sync(triage_agent, "Who was the first president of the United States?")
print(result.final_output) # Triage automatically hands off to History
```

EXAM ALERT — Handoff Latency Invariant: A key performance objective on the exam is understanding that **agent handoffs execute in 0 milliseconds** [cite: 111, 112]. Under the hood, the triage agent's handoff represents a local, in-memory reference pointer transition within the SDK's state machine [cite: 111, 112]. The transfer itself *never triggers a new network-bound LLM API call*, preserving execution speed [cite: 111, 112].

5. Guardrails, Safety & Observability Tracing

Since agentic behavior is driven by non-deterministic generative LLMs, enterprise safety requires hard-coded policy rules that override model reasoning when needed [cite: 104, 105, 107]. The OpenAI Agents SDK implements this boundary using **Guardrails** and **Observability Tracing** [cite: 104, 105, 106].

Input Guardrails (@input_guardrail) vs. Output Guardrails

Input guardrails run **before** the main agent reasoning begins, validating incoming prompts against prompt injections, malformed data, and off-topic domain requests [cite: 105, 107, 108]. Output guardrails run **after** model completion, sanitizing responses against PII data leaks, brand violations, or hallucinations [cite: 105, 107, 108].

User Prompt [] [Input Guardrail] [] (Passed) [] [Agent ReAct Loop] [] [Output Guardrail] [] (Passed) [] Output []
 (Tripwire hit) (Tripwire hit) ▼ ▼ Halt Execution Halt Execution (Exception Triggered) (Exception Triggered)

Pydantic Schema Validation & Tripwires:

A **tripwire** is a specific exception triggered by a guardrail checker that stops execution immediately [cite: 105, 107]. By leveraging **Pydantic schemas**, checker agents are forced to return strongly-typed structured outputs rather than loose conversational text, allowing deterministic tripwire evaluations [cite: 105, 107, 108].

```
from pydantic import BaseModel
from agents import Agent, Runner, input_guardrail

# Enforce structured data typing via Pydantic
class SupportCheck(BaseModel):
    is_support_related: bool

# Configure dedicated classification agent
checker_agent = Agent(
    name="Support Checker",
    instructions="Return is_support_related=True if prompt is support related, else False.",
    output_type=SupportCheck,
    model="gpt-4.1"
)

# Annotate input guardrail function with tripwire logic
@input_guardrail
def support_perimeter_guard(ctx, agent, input_text):
    check_result = Runner.run_sync(checker_agent, input_text)
    if not check_result.final_output.is_support_related:
        # Tripwire Triggered: execution halts immediately
        return {"tripwire_triggered": True, "message": "Query blocked: off-topic."}
    return {"tripwire_triggered": False}
```

Built-in Observability: Agent Tracing

The Agents SDK features native, zero-overhead tracing enabled by default [cite: 106, 108]. Traces are logged automatically to the OpenAI Developer Dashboard (platform.openai.com/dashboard/traces) [cite: 106, 107]. It captures chronological agent call sequences, exact tool parameters and observations, timestamps of handoff events, guardrail tripwire status, and step-by-step token usage [cite: 106, 107].

EXAM ALERT — Production Latency & Optimization: Tracing telemetry reveals a crucial production engineering insight: **over 99% of total runtime latency is consumed by network round-trips to the LLM API (POST /v1/responses)** [cite: 112]. Local guardrail functions, handoff pointer transitions, and Python tool scripts run in single-digit milliseconds [cite: 110, 112]. Therefore, to optimize agent speed, developers must focus entirely on *minimizing the count and payload size of LLM round-trips*, not optimizing local code execution [cite: 112].

6. Capstone Architecture Walkthrough

To ground all Module 4 concepts, the curriculum walks through a complete, multi-agent customer support project simulating an online retailer like Amazon [cite: 110, 115, 118]. This system ties together: multi-agent handoffs, custom local function tools, built-in hosted search tools, Pydantic guardrails, and tracing [cite: 110, 115, 118].

```
User Input [REDACTED] Support-Only Input Guardrail [REDACTED] Classifies query topic [REDACTED]
(SupportTopicCheck Pydantic) [REDACTED] via checker agent [REDACTED] [is_support_question == True] [REDACTED]
[REDACTED] Customer Support Triage [REDACTED] Frontline router [REDACTED] Router Agent (GPT-5.5) [REDACTED] making
handoffs [REDACTED] Handoff 1 [REDACTED] Handoff 3 [REDACTED]
[REDACTED] Handoff 2 [REDACTED]
[REDACTED] Order Status Agent [REDACTED] Refund Agent [REDACTED] FAQ Agent [REDACTED]
[REDACTED] Custom Function Tool: [REDACTED] Custom Function Tool: [REDACTED] Hosted
Tool (OpenAI): [REDACTED] • lookup_order [REDACTED] • process_refund [REDACTED] • {"type": "web_search"} [REDACTED] (Simulated Database) [REDACTED] (Simulated
Database) [REDACTED] (Live Web Retrieval) [REDACTED]
```

The Capstone Components Explained:

1. **The Mock Data Tier:** An in-memory dictionary representing customer orders (e.g. ORD-001) [cite: 115, 118]. In a production system, this tightly-coupled simulation is replaced with standard **Model Context Protocol (MCP)** servers, exposing remote database endpoints over JSON-RPC [cite: 115, 118, 119].
2. **Order Status Agent (Function Tool):** Equipped with a local Python tool decorated via `@function_tool` to query status parameters [cite: 112, 115, 118]. Runs locally in 2 milliseconds [cite: 112].
3. **Refund Agent (Function Tool):** Processes cancellations using custom python business logic [cite: 112, 115, 118].
4. **FAQ Agent (Hosted Platform Tool):** Answers policy questions by querying live internet pages via OpenAI's managed {"type": "web_search"} hosted tool, requiring zero custom local python logic [cite: 112, 115, 118].
5. **Input Guardrail:** Automatically validates support topic intent at the perimeter [cite: 112, 115, 118]. Trips the tripwire instantly if an off-topic query (e.g. 'write a poem about cats') is submitted [cite: 116, 118].

Fulfillment Logic Walkthrough:

- **User Query:** "Where is my order ORD-001?" [cite: 110, 112, 116]
- **Step A (Perimeter Guard):** Input guardrail invokes the topic checker [cite: 110, 112, 116]. Returns is_support_question = True [cite: 110, 112, 116]. Allowed through [cite: 110, 112, 116].
- **Step B (Triage):** Triage agent inspects the prompt, matches intent, and executes an in-memory control handoff to the *Order Status Agent* [cite: 110, 111, 112, 116].
- **Step C (Execution):** Order Status Agent takes control, calls lookup_order(order_id='ORD-001') [cite: 112, 116, 118], queries the database, feeds the text observation back to the LLM [cite: 112, 118], and synthesizes a natural, friendly response: "Your Wireless Headphones shipped on March 22." [cite: 112, 116, 118]

Evolving to Production Architecture: To scale this sandbox into a high-security enterprise support pipeline: (1) Replace in-memory dictionaries with remote databases via **MCP Servers** [cite: 118, 119]; (2) Layer *Output Guardrails* to sanitize generated text against brand guidelines and PII data leaks [cite: 105, 108, 125]; (3) Deploy *uvx* to run verified, vendor-published tools in ephemeral isolated containers [cite: 51, 61, 80].

7. OCI 1Z0-1157-26 Exam Cheat Sheet (Module 4)

This reference table compiles the non-negotiable architectural rules, system invariants, and core terminology for Module 4 of the OCI Agentic AI certification exam [cite: 1, 3, 85].

Exam Objective / Concept	Official Architectural Rule & Invariant (Memorize These!)
API Stack Hierarchy	The Agents SDK is a high-level Python framework built directly on top of the Responses API. It does not replace it[cite: 81, 84, 85].
Responses API Role	Serves as OpenAI's core modern interface. Replaces the legacy Chat Completions and Assistants APIs[cite: 90, 91].
Server-Side State	Responses API uses previous_response_id to chain turns server-side, avoiding client token inflation[cite: 88, 90].
The LLM Invariant	MCP and SDK wrapper protocols change tool connectivity, NOT model reasoning. The LLM remains completely protocol-unaware[cite: 61, 62, 77, 80].

Exam Objective / Concept	Official Architectural Rule & Invariant (Memorize These!)
Loop Ownership	In the Responses API, the developer manually writes/manages the loop; in the Agents SDK, the Runner manages the loop[cite: 83, 85, 100].
Hosted Tools	Platform-managed services (web_search, file_search, code_interpreter) configured with a single line[cite: 96, 99].
@function_tool Rules	Auto-compiles schemas from Python type hints and docstrings. LLM reads docstrings to choose when to run tools[cite: 96, 99, 100].
SDK Tool Categories	Three native types: Hosted Tools, Function Tools (@function_tool), and Agents as Tool (manager patterns)[cite: 96, 99, 100].
Handoff Mechanics	Decentralized pattern handing off complete context and control. Handoffs are local pointer swaps taking 0 ms[cite: 103, 113, 129].
Input Guardrails	Decorated with @input_guardrail. Run BEFORE reasoning starts to intercept injection or off-topic queries[cite: 105, 107, 108].
The Tripwire Concept	A hard-coded policy rule that raises an immediate exception to halt the agent before downstream steps execute[cite: 105, 107, 108].
Tracing Console Location	OpenAI Dashboard under platform.openai.com/dashboard/traces. On by default with zero extra setup[cite: 106, 107, 129].
Latency Bottleneck	Over 99% of wall-clock latency is caused by round-trips to the LLM API, NOT local Python tools or SDK code[cite: 112, 129].

Cross-Framework Comparison: LangChain vs. OpenAI Agents SDK

Both frameworks automate multi-step ReAct reasoning loops, but they partition their architectures differently [cite: 100, 130]:

Operational Step	LangChain Framework	OpenAI Agents SDK
1. Model Setup	Requires explicit instantiation via <code>init_chat_model()</code> before creating the agent[cite: 100, 130].	Configured directly within the single Agent constructor (<code>model="gpt-5.5"</code>)[cite: 100, 130].
2. Tool Declaration	Uses <code>@tool</code> decorator, compiling schemas from docstrings and type hints[cite: 100, 130].	Uses <code>@function_tool</code> decorator or direct platform-hosted tool dictionary entries[cite: 100, 130].
3. Agent Compilation	Built on top of a larger, composable framework (Chains, LCEL, LangGraph state nodes)[cite: 100, 130].	Consolidates model, tools, instructions, and schemas into a lightweight Agent definition[cite: 100, 130].
4. Invocation Pattern	Executed using message-based dictionaries: <code>agent.invoke({"messages": [...]})</code> [cite: 100, 130].	Clean, opinionated runner execution: <code>Runner.run_sync(agent, query)</code> or <code>await Runner.run()</code> [cite: 100, 130].
5. Output Retrieval	Often requires parsing nested message arrays: <code>response["messages"][-1].content</code> [cite: 100, 130].	Direct property access on returned result: <code>result.final_output</code> [cite: 100, 130].

8. Module 4 Practice Certification Questions

These realistic multiple-choice practice questions are modeled directly after the **Oracle Cloud Infrastructure (OCI) Agentic AI Foundations Associate (1Z0-1157-26)** certification targets for Module 4 [cite: 1, 3, 27]. Use them to self-evaluate before reviewing the answers.

Question 1: An enterprise developer is deciding whether to implement their new multi-step agent "nervous system" using the raw **OpenAI Responses API** or the **OpenAI Agents SDK**. Which of the following architectural assertions correctly identifies the trade-off in loop management?

- A) The Responses API provides automated loop management, while the Agents SDK requires the developer to manually write the loop.
- B) Both layers require manual loop construction; however, the Agents SDK provides built-in Pydantic state synchronization.
- C) The Responses API requires the developer to manually write and control the ReAct execution loop, while the Agents SDK provides an automated loop managed by a Runner engine.
- D) The Agents SDK compiles local Python tools into remote MCP endpoints, eliminating the need for an execution loop.

Question 2: When developing custom local tools for an agent using the OpenAI Agents SDK, what role do Python parameter type hints (e.g., `order_id: str`) and function docstrings play during client-to-server initialization?

- A) They are used solely as source documentation and are ignored by the underlying model reasoning engine.
- B) The SDK inspects and compiles them into a structured JSON schema, which the LLM reads to decide WHEN to call the tool and WHAT arguments to pass.
- C) They instruct the client to bypass the local subprocess and execute the function directly within the cloud database memory.
- D) They act as an output guardrail, validating that the generated natural language response matches brand guidelines.

Question 3: Under the hood, how does the modern OpenAI Responses API handle multi-turn **Conversation Chaining** without requiring the client to serialize and re-transmit the entire conversation history on every request?

- A) It stores the full history locally in the client application's SQLite database.
- B) It requires the developer to manually serialize and compress past turns into a binary blob.
- C) The client passes the previous turn's `previous_response_id`, allowing OpenAI to walk back the back-referenced chain on the server side.
- D) It relies on an active Model Context Protocol (MCP) server to store conversational history as read-only resources.

Question 4: An OCI engineer observes that an end-to-end multi-agent support query takes 11.4 seconds to execute. Tracing reveals that the custom Python tools ran in 2 milliseconds, while four calls to `POST /v1/responses` consumed the remainder of the time. Based on this trace telemetry, which production optimization strategy is most effective?

- A) Rewrite the local custom Python tool code to optimize loop execution and memory variables.
- B) Eliminate local process communication by replacing the stdio transport with Streamable HTTP ports.
- C) Focus on minimizing the count and context size of LLM API round-trips, potentially using smaller, faster models for triage tasks.
- D) Disable input guardrails to allow the triage agent to reason in parallel with the database.

Question 5: During a customer support triage session, the user enters the prompt: *"Can you write a poem about my dog?"*. The support agent's perimeter input guardrail checks this and immediately halts the execution loop with an off-topic rejection message. What is the technical term for this safety primitive?

- A) Parallel Execution Guard
- B) Tripwire Exception
- C) Autonomous Model-Driven Handoff
- D) Stateless Context Chaining

9. Practice Questions Answer Key & Explanations

Review these comprehensive technical justifications to solidify your understanding of Module 4 concepts and prepare to pass the OCI 1Z0-1157-26 certification exam.

Question 1 Answer: C

Technical Explanation: Loop ownership (the nervous system) represents the primary architectural differentiator between "building" on Layer 2 and "driving" on Layer 3 [cite: 83, 85]. The Responses API represents a low-level, direct model-completions interface where the developer must write the while loop, parsing structures, retry logic, and state transitions themselves [cite: 82, 85, 128]. The Agents SDK provides a high-level Python framework where the Runner engine automates loop executions, tool dispatching, and state management out of the box [cite: 82, 85, 128].

Why Distractors are Wrong: A is backward (the SDK automates the loop; the API is manual) [cite: 85, 128]. B is incorrect because the SDK automates the loop, meaning the developer does not build it [cite: 85, 128]. D is incorrect because MCP is a separate context protocol, and compilation does not eliminate the need for an orchestration loop [cite: 44, 46].

Question 2 Answer: B

Technical Explanation: Both LangChain and the OpenAI Agents SDK use Python reflection to parse decorated tool functions [cite: 70, 73, 96]. The SDK automatically reads the parameter type hints to compile the strict JSON parameter schemas (instructing the model what arguments and data types to pass) and converts the function docstring into the tool's machine-readable description (allowing the LLM to dynamically evaluate when and why the tool should be called) [cite: 70, 73, 96, 99].

Why Distractors are Wrong: A is incorrect because type hints and docstrings are highly functional instructions read by the AI, not just documentation [cite: 70, 73, 96]. C is incorrect because local function tools execute locally on the host, not inside a remote cloud database [cite: 73, 96]. D describes output guardrails, which are post-execution safety boundaries, not pre-execution tool schemas [cite: 107, 108].

Question 3 Answer: C

Technical Explanation: In the legacy Chat Completions API, clients had to manage context on the client side, serializing and re-transmitting the entire conversational array with every turn, which caused massive O(N) token inflation and latency [cite: 88, 90]. The modern Responses API features native server-side state retention [cite: 90]. The client simply passes the preceding turn's previous_response_id pointer, and OpenAI's server resolves the context chain dynamically, minimizing bandwidth and client-to-server payload sizes [cite: 88, 90].

Why Distractors are Wrong: A and B describe obsolete client-side state management patterns [cite: 88, 90]. D describes MCP, which is an open standard for tool and resource integration, not the Responses API's server-side conversational context database [cite: 35, 90].

Question 4 Answer: C

Technical Explanation: Tracing telemetry uniformly proves that local Python tool files, guardrail checks, and handoff pointer swaps run locally in single-digit milliseconds [cite: 110, 112]. Over 99% of wall-clock latency in agent execution is consumed by the network round-trips to the LLM API (POST /v1/responses) [cite: 112]. Therefore, optimizing the local Python code (A) yields near-zero speedups [cite: 112]. To optimize production latency, engineers must reduce the quantity and size of LLM round-trips [cite: 112].

Why Distractors are Wrong: A is incorrect because local code execution is already near-instantaneous (2 ms) compared to LLM latency (~11.4s) [cite: 110, 112]. B is incorrect because stdio process pipes are local and have zero network overhead; replacing them with HTTP sockets increases latency [cite: 55, 58]. D violates safety boundaries and does not reduce overall API calls [cite: 107].

Question 5 Answer: B

Technical Explanation: In the OpenAI Agents SDK, a guardrail function can halt execution before reaching primary agent logic by triggering a **tripwire** [cite: 105, 107]. A tripwire is a specific programmatic exception that intercepts and halts the execution loop immediately upon a safety check failure, preventing further token usage or unsafe tool actions [cite: 105, 107, 108].

Why Distractors are Wrong: A describes execution modes (parallel vs. blocking) [cite: 107]. C describes handoffs, which are decentralized control handovers [cite: 103, 129]. D describes the Responses API's conversation linking mechanic [cite: 90, 130].